

THPAELO

SEHODI

ST10195278

PART1 POE

EAPD7111

1. introduction

Currently, TechMove Logistics uses a disjointed legacy system that consists of manual communication, emails, and spreadsheets. Inefficiencies including inconsistent data, misplaced invoices, and compliance issues have arisen as a result. A Global Logistics Management System (GLMS) is suggested as a solution to these problems. Using contemporary architectural concepts, this system will be a distributed, scalable, enterprise-grade online application.

The Zachman Framework is utilized to efficiently organize the system. By examining data, procedures, and network elements from many angles, this framework offers an organized method for defining system design.

2. Zachman Framework Application

The Zachman Framework focuses on three main questions: What (Data), How (Function), and Where (Network) and divides system architecture into viewpoints (Planner and Designer) (LeanIX, n.d.).

	Data (What)	Function (How)	Network (Where)
Planner (Scope)	Shipment data Customer details- Vehicle info	Deliver goods- Track shipments- Manage Logistics operations	Cities served warehouses delivery routes

Designer (System Model)	Data tables (Customer, Orders, Vehicles, Driver)- Data relationships	System processes (Ordering Processing Tracking system, route optimizing)- Algorithms for delivery schedule	System architecture (cloud servers)GPS tracking system web access points
-------------------------	---	---	--

The method concentrates on high-level business objectives at the Planner level. Global delivery coordination, contract administration, and centralized control over shipment data are all necessary for TechMove Logistics.

The system is converted into a technical solution at the designer level. Processes are automated, data is organized into relational tables, and scalability and worldwide access are made possible via cloud infrastructure.

2.1 Planner Viewpoint (Scope Level)

The system's business view is the main focus at the planner level.

Data (What):

Important company data, including cargo details, customer information, contracts, invoicing, and driver records, will be managed by the GLMS. This guarantees the centralization of all logistics activities.

Function (How):

Core business functions including tracking shipments, handling financial transactions, processing service requests, and managing contracts will all be supported by the system.

Network (Where):

Due of TechMove's worldwide operations, the system must accommodate various locations, such as cities, warehouses, and international transportation routes.

Explanation:

Understanding the needs of the business is more important at this stage than technical details. The system makes sure that every logistics procedure is combined into a single, cohesive platform.

2.2 Designer Viewpoint (System Model Level)

The system is converted into a technical model at the designer level.

Data (What):

Customers, contracts, service requests, vehicles, and invoices are examples of the entities into which data is organized. Relational database design is used to define relationships between these elements (LeanIX, n.d.).

Function (How):

The automated procedures in the system include:

Validation of contracts (useful only for active contracts)

Processing of service requests

GPS-based shipment tracking

Algorithms for route optimization

Network (Where):

Cloud infrastructure, such as Microsoft Azure, will be used to deploy the system, allowing for: Worldwide accessibility Web and mobile interfaces Integration with GPS and external APIs

Explanation:

This stage outlines the system's actual construction, including database architecture, system logic, and infrastructure (LeanIX, n.d.).

Design Patterns Selection

Three Gang of Four (GoF) design patterns are used to guarantee scalability and maintainability.

3.1 Factory Pattern

Different kinds of service requests are created using the Factory Pattern.

Application:

Shipping Request Invoice Request Contract Renewal Request

Advantage: This pattern increases flexibility and scalability by enabling the system to construct objects without defining specific classes (Sommerville, 2016).

2. Observer Pattern

Contract status changes are tracked using the Observer Pattern.

Application:

Several parties are informed when a contract's status changes, such as when it expires:

Logistics manager, finance system, and compliance system

Advantage:

This minimizes manual monitoring and guarantees real-time updates (Sommerville, 2016).

3.3 Strategy Pattern

Currency conversion is done using the Strategy Pattern.

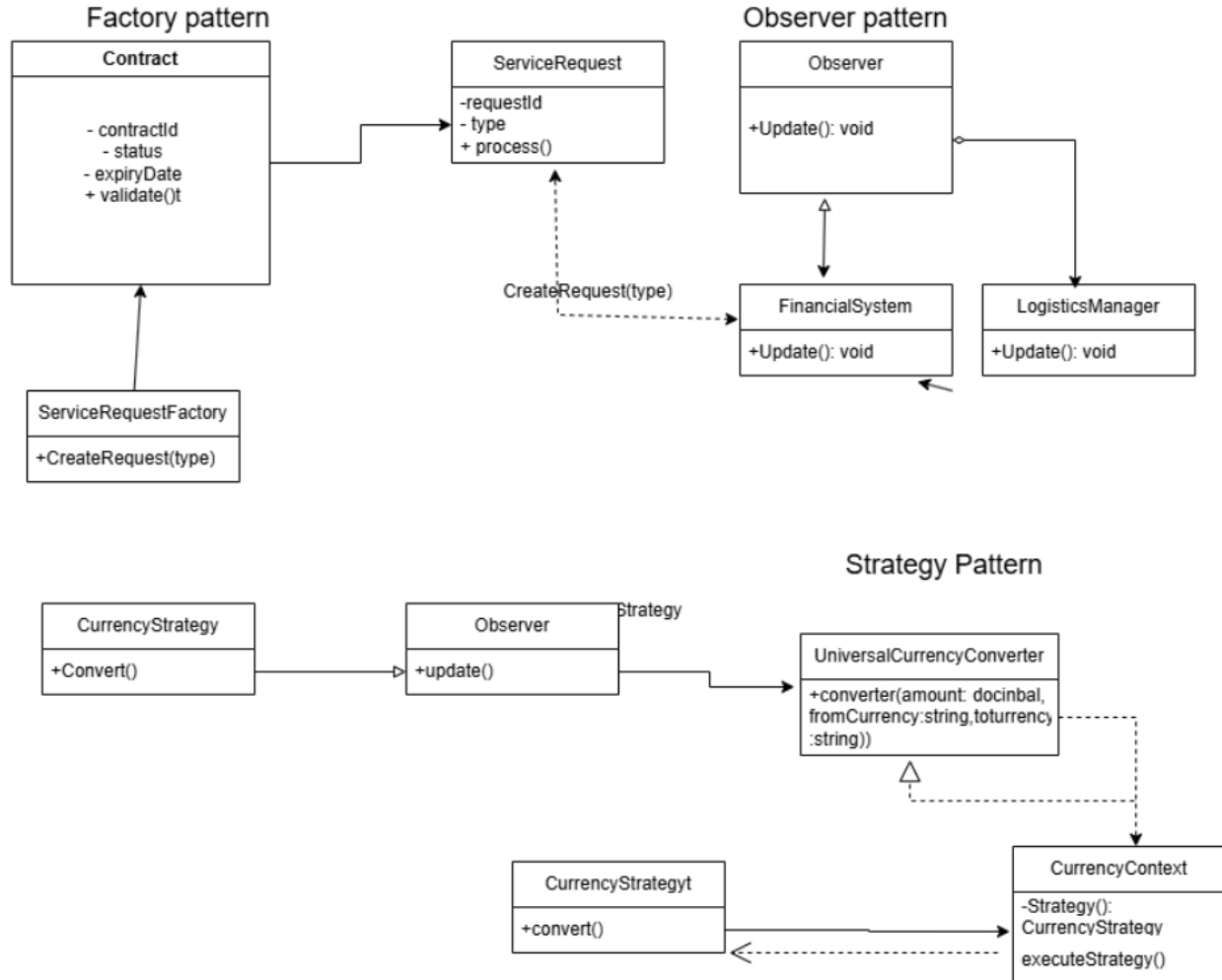
Application:

Currency conversion (USD, EUR, etc.) can be done using several algorithms or APIs.

Advantage:

permits switching between several conversion techniques without changing the fundamental logic of the system (Sommerville, 2016).

3 UML Class Diagram



Draw.io (2026) was used to design the UML.

A Detailed Explanation of the UML Class Diagram

The Global Logistics Management System (GLMS) structural architecture is represented by the UML class diagram, which uses three important design patterns—Factory, Observer, and Strategy—to show how various components work together. Software engineers frequently employ these design principles to increase the scalability, maintainability, and flexibility of systems (Sommerville, 2016).

Essential Courses

The Contract and ServiceRequest classes, which stand in for the system's primary business entities, are at the center of the figure.

ContractId, status, and expiryDate are among the characteristics included in the Contract class. Additionally, it has a method called validate() that makes sure service requests can only be created using active contracts. Maintaining data integrity and upholding system business rules depend on this validation procedure (Sommerville, 2016).

Logistics managers' requests for operations are represented by the ServiceRequest class. It has a process() method that manages execution in addition to attributes like requestId and type. Contract and ServiceRequest have a one-to-many relationship, which means that a single contract may be linked to several service requests. Multiple services can be connected to a single agreement in real-world logistical operations, which is reflected in this connection (Sommerville, 2016).

The ServiceRequestFactory class is used to implement the Factory Pattern. The createRequest(type) function of this class is in charge of dynamically creating various kinds of service requests (Sommerville, 2016).

The Contract and other system components communicate with each other via the Observer Pattern.

The GLMS uses the Strategy Pattern to manage currency conversion (Sommerville, 2016).

1. Factory Pattern (left side) Classes in your diagram:

- ServiceRequest
- Contract
- ServiceRequestFactory

What it accomplishes:

produces objects without requiring you to use new.

2. Observer Pattern (top right)

Classes:

- Observer (interface)
- FinanceSystem
- LogisticsManager

What it accomplishes:

alerts several systems when a change occurs.

3. Strategy Pattern (bottom right – GLOBAL currency)

Classes:

- CurrencyStrategy (interface)
- UniversalCurrencyConverter
- CurrencyContext

What it accomplishes:

manages the dynamic conversion of any currency

4. Non-Functional Requirements

4.1 Availability

High availability (99.9% uptime) must be attained by the system.

Application:

Microsoft Azure cloud hosting

Servers that are redundant

Automatic failover mechanisms

Description: This guarantees that the system continues to function even in the event of a failure (Sommerville, 2016).

4.2 Interoperability

External services must be integrated with the system.

Execution:

JSON/XML data types for REST APIs

Integration with GPS and currency exchange systems

Explanation:

This makes it possible for GLMS to effectively communicate with other systems.

5. Scalability Strategy

The system must effectively manage the extra strain if TechMove adds 50,000 new contracts (Sommerville, 2016).

5.1 Horizontally Scaling

Several servers are added rather than upgrading just one.

Advantage:

enhances efficiency and manages heavy traffic (Sommerville, 2016).

5.2 Loads Balancing

Multiple servers are used to spread traffic.

Advantage:

ensures dependability and avoids system overburden (Sommerville, 2016).

5.3 Caching

Data that is often accessed, such as contracts and currency rates, is temporarily stored.

Advantage:

enhances response time and lessens database load (Sommerville, 2016).

5.4 Microservices & Containerisation

The system is separated into separate services:

Contract, Logistics, and Payment Services

Docker containers are used to run each service (Sommerville, 2016).

Advantage:

Simpler scaling

Self-sufficient deployment

Improved fault isolation

6. Conclusion

TechMove Logistics is transformed from a disjointed old system into a cutting-edge, scalable corporate solution by the suggested GLMS system. The Zachman Framework is used to clearly structure the system from a technical and business standpoint. While cloud-based deployment and scalability solutions ensure great performance under growing demand, the usage of design patterns guarantees flexibility and maintainability.

Reference List

LeanIX, "The Zachman Framework – A Definitive Guide." [Online]. Available: <https://www.leanix.net/en/wiki/ea/zachman-framework>. [Accessed: Apr. 13, 2026].

diagrams.net (n.d.) Diagrams.net (draw.io) – Online diagramming tool. Available at: <https://app.diagrams.net/> (Accessed: 07 April 2026).

Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.